

SpaceSails: Secretly a Classroom

Deterministic Real-Time Orbital Simulation in the Browser — and the Human-PO / AI-Head-Coder Method That Built It

Erno Soinila, D.Sc. (Eng.)

Senior AI Engineer, Efima — Product Owner

`erno.soinila@efima.com`

Claude (Fable 5)*

Head Coder

Anthropic

DRAFT v0.2 — July 2026

Abstract. SpaceSails is a browser-native solar-system sailing and piracy game whose engine is a deterministic 2D N -body integrator compiled to WebAssembly. Nothing in it cheats: every trajectory is integrated, every prediction is honestly uncertain, and the hardest maneuvers are hard because physics says so. The same properties that make the game honest make it a classroom: the repository ships a thirteen-lesson numerical-methods lab course that runs on the game’s own engine, and the game’s newest mechanic — fire control — is the shooting method for boundary value problems, played. We describe (1) the system: a determinism-first core, adaptive symplectic integration with exact maneuver-node landing, honest uncertainty cones, and a closed-form no-tunneling proximity test; (2) the duty-station interface that distributes one simulation across eight crew desks; and (3) the process: a two-person team — a human product owner and an AI “head coder” — that took the project from empty repository to roughly sixty reviewed, tested, merged pull requests in four days, with the owner approving PRs from a phone and the AI implementing, verifying in a live browser, and managing the merge train. We argue the combination — deterministic simulation as both gameplay substrate and curriculum, plus AI-driven implementation under tight human product direction — is a repeatable method, and we report what it cost, where it failed, and what the failures taught the game.

1 Introduction

On the surface, SpaceSails is a piracy game: solar sails, cargo pods, hunters, a gun deck. Underneath, it is a numerical-methods and orbital-mechanics classroom, and the disguise is deliberate. “Secretly edutainment” is the project’s design law, not an accident of scope: the owner’s first shipped software (SoittoPeli, 1998) was music edutainment for children, and SpaceSails carries the same DNA on purpose.

Our thesis is that *honesty* is the unifying design constraint, and that it pays for itself three times over. The simulation never lies: it is bit-deterministic, so replays, tests, and lessons are the same artifact (§2). The instruments never lie: prediction cones admit exactly what they do not know, and that admission is itself a game mechanic (§5.2). And the curriculum never lies: every number printed in the lab course comes from running that lesson’s probe against the shipping engine, which makes the course an adversarial test suite the engine must survive (§5). Play the game well and you have quietly learned the real thing.

This paper contributes:

1. a determinism-first architecture for real-time orbital simulation in WebAssembly, with the concrete rules

*Listed to record the head-coder role faithfully. Publication venues (e.g. ACM) do not permit AI systems as authors — authorship requires the capacity to take responsibility for the work — so on submission this credit becomes an extended acknowledgment with the same content.

that kept client and (archived) multiplayer server bit-identical;

2. honest uncertainty as a game mechanic: prediction cones whose growth law was *falsified by the game’s own lab course* and then fixed (§5.2);
3. fire control as pedagogy: the shooting method for boundary value problems — damped Newton iteration over launch bearing and ejection charge, with every residual evaluated by one flight of the real integrator — surfaced as a visible, converging CALCULATING FIRING SOLUTION moment (§5.3);
4. an experience report of the human-PO / AI-head-coder method at unusual cadence, with the working agreements that made review at that pace possible (§6).

Because the entire project lives in one public repository built at high cadence, its git history is unusually complete: every model decision, every rejected alternative, and every owner correction is a commit message. Section 3 exploits this to give an autobiographical account of how the gravity model of §2 actually came to be.

2 The Deterministic Core

2.1 Determinism is law

No wall clock, no Random, no environment-dependent math anywhere in SpaceSails.Core. Every state transition is a pure function of the prior state and a fixed timestep schedule. The consequences compound: replays are free; the archived multiplayer server agreed with clients bit-for-bit; and — decisive for this paper — *tests and lessons are the same artifact*. A lab probe is just a test that prints its evidence.

2.2 Dynamics and integration

Celestial bodies ride rails: an ephemeris interface gives position as a pure function of time, $\mathbf{r}_k(t)$. Ships are test particles feeling point-mass gravity from every body,

$$\mathbf{a}_i(\mathbf{r}_i, t) = \sum_k \mu_k \frac{\mathbf{r}_k(t) - \mathbf{r}_i}{\|\mathbf{r}_k(t) - \mathbf{r}_i\|^3}, \quad \mu_k = Gm_k, \quad (1)$$

integrated with semi-implicit (symplectic) Euler:

$$\begin{aligned} \mathbf{v}_{n+1} &= \mathbf{v}_n + \mathbf{a}(\mathbf{r}_n, t_n) \Delta t_n, \\ \mathbf{r}_{n+1} &= \mathbf{r}_n + \mathbf{v}_{n+1} \Delta t_n. \end{aligned} \quad (2)$$

The velocity-first update is what buys long-horizon energy behavior at first-order cost: the scheme is symplectic, so orbits precess rather than spiral, and a multi-week projection remains a trustworthy *shape* even where it is not a trustworthy *ephemeris*.

2.3 Adaptive stepping with exact node landing

The step size follows the local dynamical time of the nearest dominant attractor,

$$\Delta t_n = \text{clamp}\left(\eta \min_k \sqrt{\frac{d_{ik}^3}{\mu_k}}, \Delta t_{\min}, \Delta t_{\max}\right), \quad (3)$$

where $d_{ik} = \|\mathbf{r}_k - \mathbf{r}_i\|$ and η is a safety fraction: small steps in deep gravity wells, long strides in interplanetary cruise. One further rule makes plotted maneuvers exact rather than approximately timed: the step is truncated to land precisely on every maneuver node's timestamp,

$$\Delta t_n \leftarrow \min(\Delta t_n, t_{\text{node}} - t_n), \quad (4)$$

so a plotted burn executes in the projection at the same instant the live loop fires it. Universal-variable Kepler propagation and patched conics were rejected on purpose: they assume one attractor per arc and disagree with the integrator exactly where the game happens — flybys. (Lab 06 §C quantifies what adaptive stepping still misses at periapsis and what it would cost to fix.)

2.4 The no-tunneling rule

Closest-approach queries and ordnance-hit tests never sample: within each integrator step the relative motion of two objects is piecewise linear, $\mathbf{p}(s) = \mathbf{p}_0 + s\mathbf{w}$ with $s \in [0, 1]$, where $\mathbf{p}_0$ is the relative position at the start of the step and $\mathbf{w} = \Delta\mathbf{v}\Delta t$ the relative displacement across it. The minimum distance has a closed form:

$$s^* = \text{clamp}\left(-\frac{\mathbf{p}_0 \cdot \mathbf{w}}{\|\mathbf{w}\|^2}, 0, 1\right), \quad d_{\min} = \|\mathbf{p}_0 + s^*\mathbf{w}\|. \quad (5)$$

Quadratic algebra, so *no step size can tunnel a fast graze through a target*. This replaced a per-sample check that the lab course itself proved dishonest (§5.2).

2.5 WebAssembly performance envelope

The client runs interpreted IL in the browser at roughly $100\times$ native cost, and the architecture is shaped by that number rather than in denial of it: NPC route search integrates at one-day steps, live NPCs advance in 60-second quanta, and the high-warp path adapts quantum length to time compression. Determinism makes these coarsenings safe to test: the coarse and fine paths are compared step-for-step in the suite.

3 Gravity at Speed in a Browser: How the Model Happened

Section 2 described the model as if it had been designed in one sitting. It was not. It accreted, one falsified assumption at a time, and the repository's git history records every step. This section replays that history in first person — the head coder's account of the thoughts that led to the model the Core runs today.

3.1 Day one: choose boring, test bitwise (M1)

The first physics commit made four decisions in an afternoon, all of them deliberately boring. Double precision and a 2D ecliptic plane, because the game is a map and doubles are what WebAssembly does natively. Planets on rails — position as a pure function of time — because planets do not care about ships, and rails make the N -body problem a one-body problem N times. Ships as test particles feeling Eq. (1). And semi-implicit Euler, Eq. (2), at a fixed one-second step.

The integrator choice deserves the honesty it was made with. Explicit Euler was never a candidate: it injects energy and orbits visibly spiral outward within days. Runge–Kutta 4 was the respectable alternative — fourth-order accurate — but it costs four gravity evaluations per step and is not symplectic, so its orbits decay *slowly*, which is worse than decaying visibly: the error hides until a two-year plot exposes it. Semi-implicit Euler is first-order, crude, and symplectic — it conserves a nearby Hamiltonian, so orbits precess instead of spiraling. For a game whose long projections must be trustworthy *shapes*, phase error is survivable and energy

error is not. The commit landed with the test that made the choice accountable: a 30-day circular orbit holding radius to $< 0.5\%$, and a bitwise-identical replay test that has guarded every physics change since.

3.2 Plotting mode breaks the fixed step (M4)

The fixed one-second step survived exactly three milestones. Plotting mode needed to draw a 60-day trajectory ribbon — later 730 days — and at $\Delta t = 1$ s that is 6×10^7 integration steps per replot, in a browser, in interpreted IL. The observation that rescued it is the oldest one in celestial mechanics: nothing happens in deep space. The local dynamical time $\sqrt{d^3/\mu}$ is the natural clock of the problem, so the step became a fixed fraction ($\eta = 1/64$) of it, clamped to $[1 \text{ s}, 1 \text{ h}]$ — Eq. (3). Coarse on a cruise, fine through a flyby, and deterministic, because the step size is a pure function of state and ephemeris.

Two details of that commit set the model’s character. First, steps land *exactly* on maneuver-node times, so a plotted burn executes in the projection at the same instant the live loop fires it — without that, plans and reality drift apart at every burn and the plotting table lies. Second, the commit message records the road not taken: universal-variable Kepler propagation and patched conics, the NASA mission-planning classic, rejected because they assume one attractor per arc and would disagree with the integrator exactly at flybys, where the game happens. The acceptance test priced the compromise: $< 5 \times 10^7$ m divergence from $\Delta t = 1$ s truth over a 20-day mid-course-burn transfer, against a capture threshold of 10^9 m.

3.3 The model learns to carry passengers (M7, M14)

The Electric Universe layer added plasma forces and hull charge, and immediately collided with the adaptive step: a naive charge-relaxation update goes unstable when Δt is an hour. The fix — a clamped exponential approach to ambient charge, $\Delta q = (q_{\text{amb}} - q) \min(1, \Delta t / \tau)$ — was the first time a feature had to be written *for* the integrator rather than merely on top of it, and it set the rule every later force followed: stable at any step the planner might take.

The owner’s playtests then calibrated what the model had to be good *for*. His standard became a Core test with his metric in its name, `SaturnSail_FitsInOneNavDeskSitDown`: one plotted plan, entered at the nav desk in one sitting, must reach Saturn’s port zone inside a 730-day horizon — and a Mercury hard-brake transfer must fit the same sit-down. The integrator’s accuracy budget is not an abstract tolerance; it is “the perihelion of the plotted ellipse kisses Mercury’s orbit ring on the owner’s screen.”

3.4 The owner’s question that merged two integrators (M19)

For eighteen milestones the game ran two integrators: the adaptive one for planning and the fixed one-second loop for living. Then the owner, mid-voyage at $10000\times$ warp, asked in one sentence what should have been obvious: “*since it already knows the trajectories from planning, why is it so heavy running?*” He was exactly right. At $10000\times$ the live loop performed 10,000 gravity-plus-plasma evaluations per real second for the player ship alone, nearly all of them in deep space where the dynamical time is measured in weeks.

The fix was to let the live simulation *become* its own projection: `RunAdaptive` — the planner’s stepping, Eq. (3), exact node landing included, returning only the final state. Above $100\times$ warp the time accumulator is consumed in fixed 60-second quanta through it; fixed quanta keep the trajectory independent of frame timing, preserving determinism. Below $100\times$, nothing changed, byte for byte. Measured headless with plasma on and full traffic: 9,969 simulated seconds per real second at $10000\times$ — effectively the full requested rate, in a browser, in interpreted IL. The projection method and the game loop have been the same code path since; matching the fixed integration within 0.1% over a 30-day cruise with a mid-course burn is a regression test.

3.5 The labs get the last word

The model’s most recent changes were forced, not chosen: Lab 06 proved the sampled closest-approach scan could tunnel through a periapsis and Lab 08 proved the uncertainty cone lied after a burn (§5.2); the closed-form segment minimum of Eq. (5) and the impulse term of Eq. (7) are the repairs. The pattern across all five subsections is the same one: every piece of the current model exists because something honest — a test, a lab, or the owner at $10000\times$ warp — caught the previous model being wrong or wasteful. Nothing in the integrator is there because it was elegant in the abstract; it is there because the $100\times$ interpreter tax and a two-year plotting horizon left no room for anything that was not.

4 One Simulation, Eight Desks

The ship is crewed, not driven. One deterministic simulation is projected onto eight duty stations: *Captain* (standing orders, ship status board), *Nav* (map, plotting table, orbit-assist autopilot), *Sensors* (telescope ledger, passive watch, the eyes race), *War room* (tactical circle, intercept clock, fire control), *Trade*, *Comms* (departures board, dark web), *Galley* (news feed, rum), and *Deck* (a walkable interior whose raycast first-person windows show the real ephemeris sky).

Two interface rules survived playtesting. First, a desk’s own topic owns roughly 70% of the screen; every other station rides along as a one-line “chip” — a current-objective summary, never a stat dump — with the captain’s chip leading everywhere. Second, honest instruments are themselves game-play: sun-glare detection asymmetry makes anti-sunward the

pirate’s hunting angle; a charged hull glows farther than it sees; and the mutual-visibility readout (“do we see them, or do they see us first?”) turns the sensor model into strategy.

5 Secretly a Classroom

5.1 The Gravity Lab

The repository ships thirteen type-it-in lessons (labs/), each a console probe referencing the game’s own `SpaceSails.Core`: motivation, the standard-textbook treatment (with Curtis pointers), what the game simplifies away, a numerical experiment with real printed output, and “break it yourself” exercises. The honesty rule is absolute: every printed number came from running that lesson’s probe; rerun and re-paste, never hand-edit.

5.2 The labs falsify the game — and the game gets fixed

The original prediction cone grew as

$$w(\Delta t) = w_0 + \sigma_v \Delta t + \frac{1}{2} a_{\max} \Delta t^2, \quad (6)$$

with w_0 the present track uncertainty, σ_v the velocity uncertainty, and $a_{\max}$ a bound on unmodeled acceleration. Lab 08 measured Eq. (6) against a target that actually burns and found it *flatly excluded the truth for hours after a real burn*: conservatism ratio $0.5 \times$ at both 2 h and 6 h — the cone claimed half the width the evidence demanded. The fix adds an impulse term growing linearly at the plausible burst Δv ,

$$w(\Delta t) = w_0 + \sigma_v \Delta t + \frac{1}{2} a_{\max} \Delta t^2 + \Delta v_{\text{burst}} \Delta t, \quad (7)$$

and shipped as a prerequisite of fire control, with the lab’s dishonesty table reproduced verbatim as a regression test.

Lab 06 caught the older sampled closest-approach scan stepping clean through a periapsis: 380,845 km reported versus 157,866 km true — a $2.4 \times$ error at exactly the moment a player cares — and priced the fix. The closed-form segment minimum of Eq. (5) is that fix, and ordnance hit resolution inherits it.

The point for this venue: the curriculum is not documentation *about* the engine; it is an adversarial test suite the engine must survive, written to be read.

5.3 Fire control as the shooting method

A firing solution is a two-point boundary value problem: leave the muzzle now, occupy the predicted target point at t_{hit} . The unknowns are launch bearing θ and ejection charge q ; the residual is the miss vector at arrival,

$$\mathbf{R}(\theta, q) = \mathbf{x}(t_{\text{hit}}; \theta, q) - \hat{\mathbf{x}}_{\text{tgt}}(t_{\text{hit}}), \quad (8)$$

where $\mathbf{x}(t_{\text{hit}}; \theta, q)$ is obtained by one full flight of the real integrator of §2 — no closed-form ballistics, no cheating.

The solver is a damped 2×2 Newton iteration,

$$\begin{pmatrix} \theta \\ q \end{pmatrix}_{m+1} = \begin{pmatrix} \theta \\ q \end{pmatrix}_m - \lambda J_m^{-1} \mathbf{R}_m, \quad J_m = \frac{\partial \mathbf{R}}{\partial (\theta, q)} \Big|_m, \quad (9)$$

with J_m estimated by finite differences (each column one more integrator flight) and $\lambda \in (0, 1]$ a damping factor that backs off when a step would overshoot. This is the shooting method, verbatim, from any numerical-methods course — and the game plays it back honestly: the gun deck replays the iteration trace one Newton step per beat (CALCULATING FIRING SOLUTION...), locks at $T - 60$ s, slews the hull, fires, and returns the helm — the Norden-bombsight beat, earned rather than faked.

Dispersion is not a tuning knob. The shot’s uncertainty is the target track’s cone half-width at arrival,

$$\sigma_{\text{hit}} = w_{\text{tgt}}(t_{\text{hit}}) \quad (10)$$

with w_{tgt} from Eq. (7), so fire-control quality *is* track quality: telescope work at the Sensors desk directly buys tighter shots at the gun deck — one weapon system distributed across two stations. Each locked solution prints its “gunner’s lesson” (lead angle, time of flight): the solver as flight instructor, teaching the same intuition manual intercepts demand.

6 Experience Report: a Human PO and an AI Head Coder

6.1 Roles

The owner sets vision, plays every build, files corrections in plain language (“the sensors are off by default — why?”), and approves pull requests, often from a phone. The AI implements, tests, *verifies live in a real browser via automation*, writes the PR narratives, and runs the merge train. Review is by playing, not by reading diffs alone.

6.2 Cadence

As of 2026-07-05: from empty repository to the full game plan (M0–M10) in one day of implementation; roughly sixty merged PRs over four days; ~ 180 deterministic Core tests (239 at the time of this revision); a thirteen-lesson lab course; and the entire fire-control feature set shipped overnight as a six-PR stacked chain reviewed at morning coffee.

6.3 Working agreements that made the pace safe

- **Determinism is law** — it is what makes review-by-playing sound, because what the reviewer plays is what the tests tested.
- **Core-first with tests**: every mechanic lands as a pure rule with a deterministic test before any UI touches it.

- **Merge-train hygiene:** one new file per parallel lane; one-line appends at marked anchors in the shared hotspot file.
- **No unverified claims:** every sentence in a PR body is verified in a live browser before it is written.

6.4 Failure gallery (honest)

The interpreted-WASM freeze class (hit twice); the destroyed-canvas class — a Blazor conditional unmounting a canvas leaves the renderer holding a dead context (found three times, then prevented by convention); the “three target locks” perception bug, where the simulation was right and the presentation lied; transient Pages deploy failures and the retry playbook; and a stacked-PR merge-train pitfall — deleting a merged base branch auto-closes its children — recovered live and folded into the plan document’s procedure.

6.5 What the AI could not do

Choose what the game should feel like. Every mechanic in §4–§5 traces to an owner sentence written in plain language. The method’s throughput is the owner’s taste executed quickly, not replaced.

7 Related Work

Kerbal-genre orbital games adopt patched conics — piecewise two-body arcs with sphere-of-influence handoffs — trading multi-attractor truth for closed-form speed; we take the opposite stance and integrate everything, accepting Eq. (3)’s cost to keep flybys honest. The educational-games literature on “stealth learning” anticipates our disguise strategy; our addition is making the curriculum executable against the shipping engine. Determinism-first architectures descend from lockstep RTS netcode traditions. Prior AI-pair-programming experience reports describe a programmer and an assistant; we report a different split — *product owner and head coder* — in which the human writes no code at all.

8 Limitations and Future Work

The simulation is a 2D ecliptic plane with on-rails circular ephemerides. Multiplayer is archived pending the economy’s server-side move; persistence is absent by design so far. Lesson 13 (“Shooting, literally”) is drafted as the fire-control lab and lands with its probe. The Ancients’ auto-plot deliberately rations automation to protect the pedagogy.

9 Summary

SpaceSails demonstrates that one uncompromising property — bit-determinism — can be spent three ways at once: as an engineering tool (replay, lockstep, testability), as a game-design

tool (honest instruments whose admissions of ignorance are mechanics), and as a pedagogical tool (a lab course that is also an adversarial regression suite). The engine’s best fixes — the impulse-aware cone of Eq. (7), the closed-form no-tunneling test of Eq. (5) — were forced by its own curriculum, and its flagship mechanic is a numerical method played at the gun deck. Around that core, a two-person human-PO / AI-head-coder team sustained roughly fifteen reviewed, merged PRs per day without lowering the review bar, because determinism made playing the build a sound review procedure. We believe both halves — the simulation-as-curriculum design and the PO/head-coder process — are repeatable, and we offer the repository itself as the evidence.

Reproducibility. Everything in this paper runs from the public repository: `dotnet test` for the evidence, `dotnet run -project labs/NN-... -c Release` for every printed number, and the game itself at the GitHub Pages deployment (<https://esoinila.github.io/SpaceSails-play>).